



**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

Centre for Professional and
Continuing Education

(SCTP) Advanced
Professional Certificate
Data Science and AI



3.8 Computer Vision (CV)

Module Overview

3.1 Introduction to Machine Learning

3.2 Probability and Statistics for Machine Learning

3.3 Supervised Learning

3.4 Supervised Learning - Advanced

3.5 Unsupervised Learning

3.6 Time Series Data & Forecasting

3.7 Neural Network & Deep Learning

3.8 Computer Vision (CV)

3.9 Natural Language Processing (NLP)

3.10 Transformers, Attention & GenAI

Where We Left Off

From session features to pixels

Marcus, end of L07:

"Can we auto-tag product photos as dress, shirt, sneaker?
We have ~10,000 new uploads each season."

What's different this week

- ▶ **New modality** Inputs are images ($28 \times 28 \times 1$ or $224 \times 224 \times 3$ tensors), not tabular rows
- ▶ **New architecture** Convolutional layers replace the flat MLP first layer
- ▶ **New tool: transfer** Pretrained models from torchvision — don't start from scratch
- ▶ **Same training loop** `zero_grad` → forward → loss → backward → step (L07 unchanged)

Today's Three Questions

From pixels to predictions

1

What does an image LOOK LIKE to a neural network?

A tensor of pixel intensities — and why MLPs choke on it.

2

What IS a convolutional neural network?

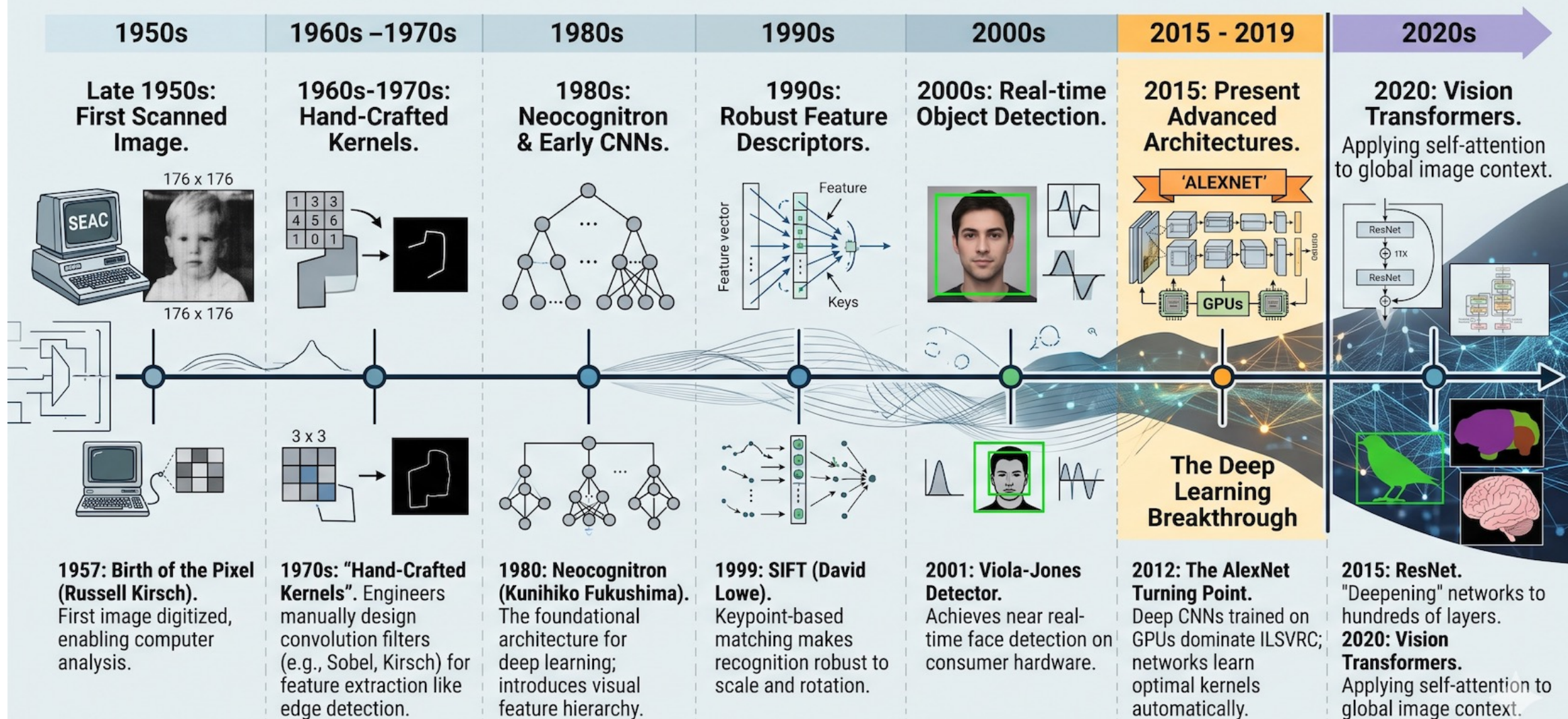
Stack of (Conv $\rightarrow$ ReLU $\rightarrow$ Pool) blocks + a classifier head.

3

When should we reach for transfer learning?

When data is small AND the domain is close to ImageNet.

'A TIMELINE OF COMPUTER VISION: FROM PIXELS TO PREDICTION'



Question 1: The Flat-MLP Problem

Why MLPs don't scale to real images

Flatten the image, feed it to a 2-hidden-layer MLP

28×28 Fashion-MNIST

Input dim	784
Hidden 1 (256)	200,960 params
Hidden 2 (128)	32,896 params
Output (10)	1,290 params
TOTAL	~235K parameters

224×224×3 (real photos)

Input dim	150,528
Hidden 1 (256)	38.5M params
Hidden 2 (128)	32,896 params
Output (10)	1,290 params
TOTAL	~38.6M params ✨

Parameter count scales LINEARLY with input size. Need a smarter approach.

The Spatial-Locality Problem

What MLPs can't exploit

Two facts about images that MLPs IGNORE

1. LOCALITY — nearby pixels are related (edges, textures live in small patches)
2. TRANSLATION INVARIANCE — a sleeve is a sleeve top-left or bottom-right

An MLP must LEARN these from data. Expensive and fragile.

What we need

A model that BAKES IN locality and translation invariance.

Same feature detector slides everywhere — fewer parameters, better inductive bias.

That model is the Convolutional Neural Network.

Question 2: What is Convolutional Neural Network (CNN)?

Each of the CNN block contains Conv → ReLU → MaxPool

Three operations, stacked the CNN block few times, then a Flatten + Linear head

Conv2d(in, out, kernel=3, padding=1)



ReLU()



MaxPool2d(2)



Fully Connected (FC) Layer



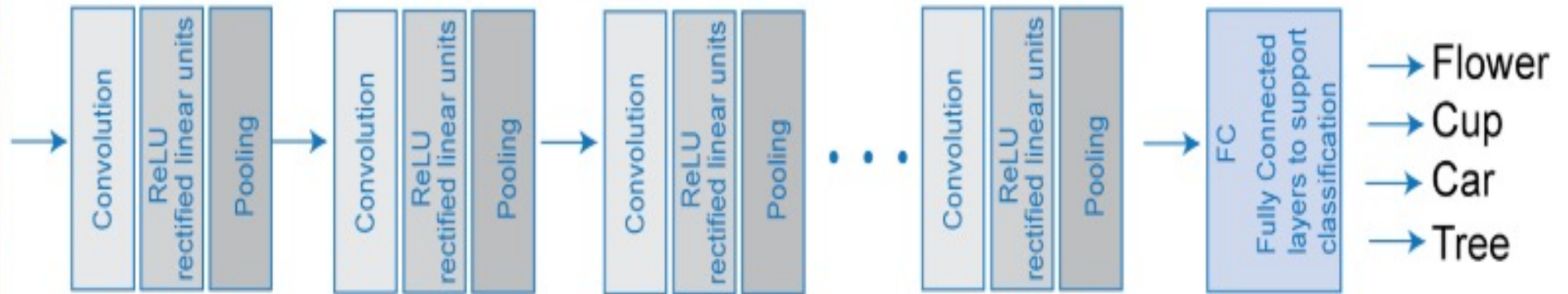
Linear

The CNN Block

Conv → ReLU → MaxPool

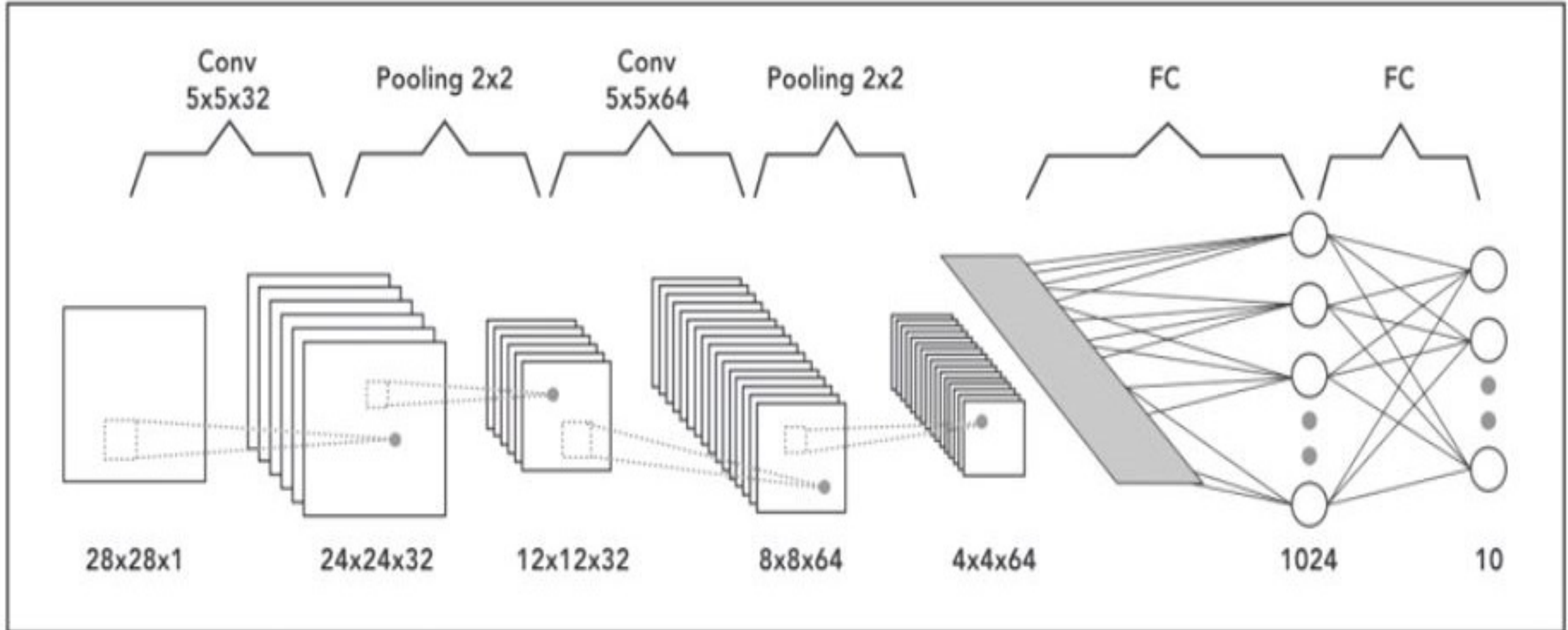


Input Image



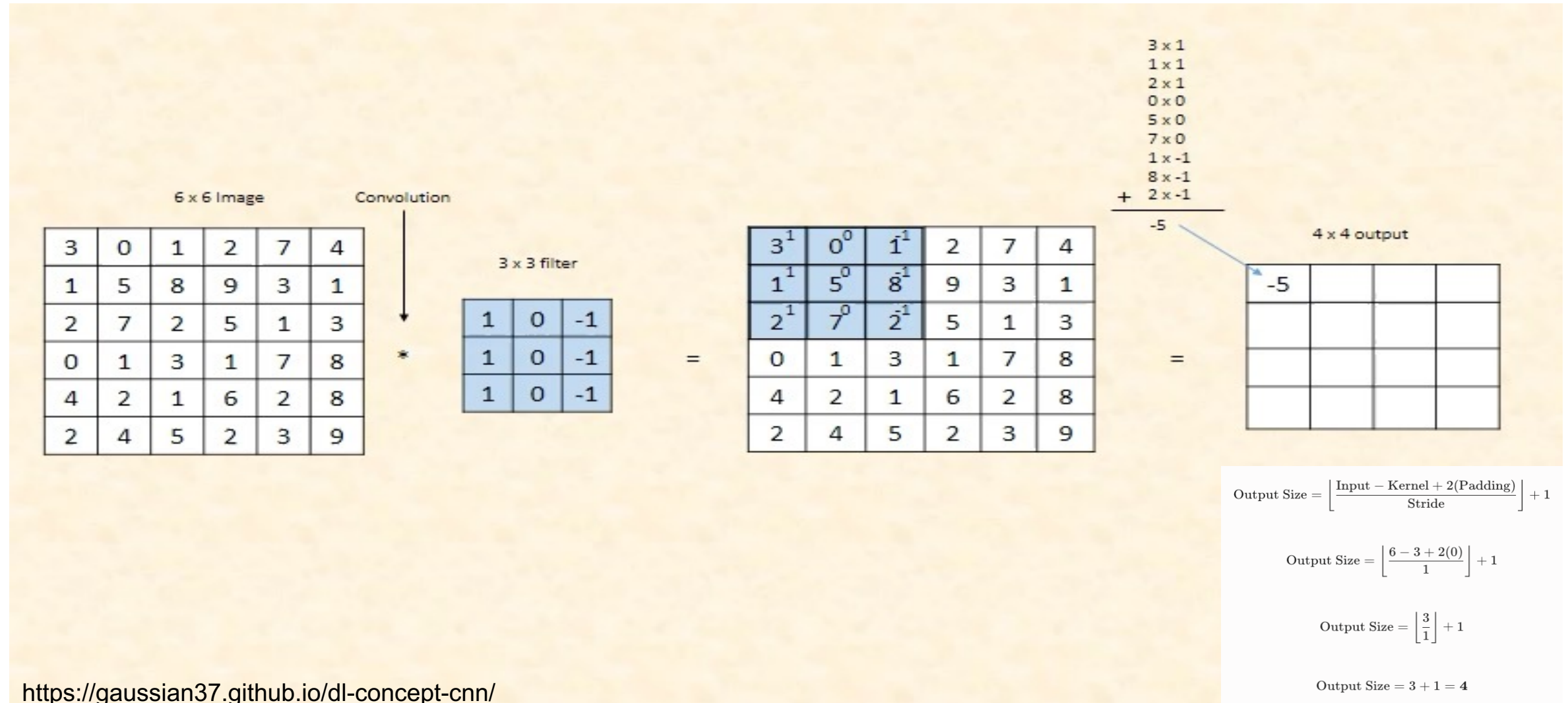
The CNN Block

Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool



Convolution Operation

Input Size = 6, Kernel Size = 3, Padding = 0, Stride = 1, Output Size = 4



<https://gaussian37.github.io/dl-concept-cnn/>

CLASSICAL EDGE DETECTION KERNELS: THE MANUALLY CRAFTED FILTERS OF COMPUTE VISION

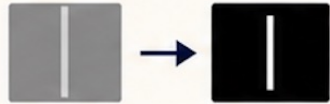
SOBEL OPERATOR (GRADIENT-BASED)

G_x (Horizontal Edge Detection)

-1	0	1
-2	0	2
-1	0	1

G_y (Vertical Edge Detection)

-1	-2	-1
0	0	0
1	2	1



Emphasizes strong edges and reduces noise

PREWITT OPERATOR (GRADIENT-BASED)

G_x

-1	0	1
-1	0	1
-1	0	1

G_y

-1	-1	-1
0	0	0
1	1	1



Soft, smoother edges

LAPLACIAN (SECOND-ORDER GRADIENT)

0	1	0
1	-4	1
0	1	0

Omnidirectional, sensitive to edges of any orientation, but highly sensitive to noise

KERNEL CONVOLUTION IN ACTION

1) Original gray-scale input image



2) Apply kernels (Sobel vs Prewitt)

Sobel

G_x

-1	0	1
-2	0	2
-1	0	1

G_y

-1	-2	-1
0	0	0
1	2	1

Prewitt

G_x

-1	0	1
-1	0	1
-1	0	1

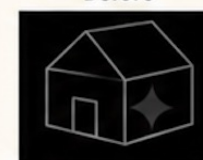
G_y

-1	-1	-1
0	0	0
1	1	1

3) Gradient magnitude output (Before vs After)

Sobel

Before



After



Prewitt

Before



After



4) Combined Gradient Magnitude (After)

Sobel



Prewitt



<https://setosa.io/ev/image-kernels/>

Padding: 0's added around input for kernel to reach edges.

Input Size = 5, Kernel Size = 3, Padding = 1, Stride = 1, Output Size = 5

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel

0	-1	0
-1	5	-1
0	-1	0

114				

$$\text{Output Size} = \left\lfloor \frac{\text{Input} - \text{Kernel} + 2(\text{Padding})}{\text{Stride}} \right\rfloor + 1$$

$$\text{Output Size} = \left\lfloor \frac{5 - 3 + 2(1)}{1} \right\rfloor + 1$$

$$\text{Output Size} = \left\lfloor \frac{4}{1} \right\rfloor + 1$$

$$\text{Output Size} = 4 + 1 = 5$$

<https://medium.com/@sushmita2310/understanding-convolutional-neural-networks-cnns-for-beginners-e85ad21fe432>

Stride: How many pixels the kernel moves at each step.

Input Size = 5, Kernel Size = 3, Padding = 1, Stride = 2, Output Size = 3

0 ₂	0 ₀	0 ₁	0	0	0	0
0 ₁	2 ₀	2 ₀	3	3	3	0
0 ₀	0 ₁	1 ₁	3	0	3	0
0	2	3	0	1	3	0
0	3	3	2	1	2	0
0	3	3	0	2	3	0
0	0	0	0	0	0	0

1	6	5
7	10	9
7	10	8

$$\text{Output Size} = \left\lfloor \frac{\text{Input} - \text{Kernel} + 2(\text{Padding})}{\text{Stride}} \right\rfloor + 1$$

$$\text{Output Size} = \left\lfloor \frac{5 - 3 + 2(1)}{2} \right\rfloor + 1$$

$$\text{Output Size} = \left\lfloor \frac{4}{2} \right\rfloor + 1$$

$$\text{Output Size} = 2 + 1 = 3$$

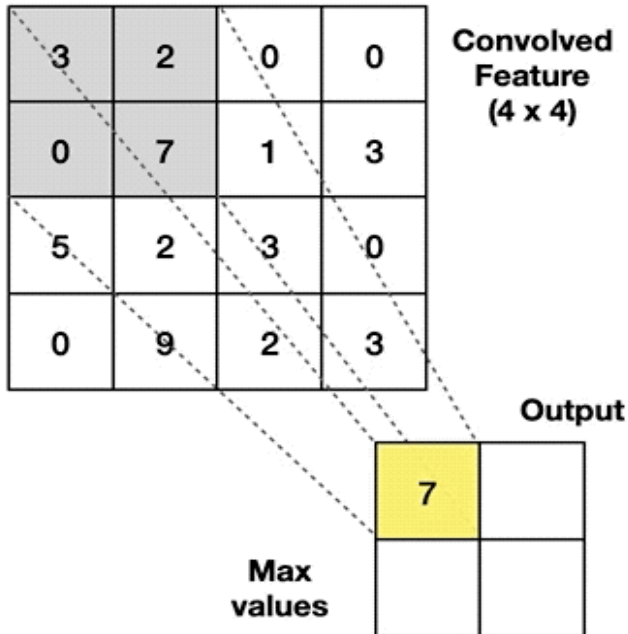
Max Pooling: Shrinks output size with translation tolerance.

Input Size = 4, Pooling Size = 2, Stride = 2, Output Size = 2

Max Pooling

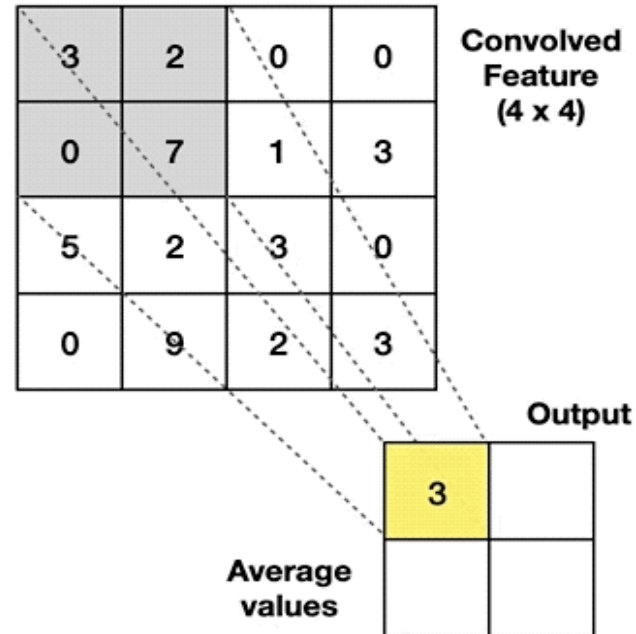
Take the **highest** value from the area covered by the kernel

Example: Kernel of size 2 x 2; stride=(2,2)



Average Pooling

Calculate the **average** value from the area covered by the kernel



$$\text{Output Size} = \left\lfloor \frac{\text{Input} - \text{Pooling Size} + 2(\text{Padding})}{\text{Stride}} \right\rfloor + 1$$

$$\text{Output Size} = \left\lfloor \frac{4 - 2 + 2(0)}{2} \right\rfloor + 1$$

$$\text{Output Size} = \left\lfloor \frac{2}{2} \right\rfloor + 1$$

$$\text{Output Size} = 1 + 1 = 2$$

CODE-ALONG

Open:

- `01_monday_morning.ipynb`
- `02_convolution_intuition.ipynb`
- `03_first_cnn.ipynb`
- `04_transfer_learning.ipynb`

90 minutes · Stop at the Extension marker

Question 3: Transfer Learning — Why?

Real catalogues have hundreds of images, not millions

The reality at NorthStar

Fashion-MNIST has 60,000 training images. Marcus's catalogue has ~500 per class.
Train a CNN from scratch on 500 images and it OVERFITS — memorises noise.

The fix — don't start from scratch

Start from a model someone else trained on a much bigger dataset (ImageNet, 1.2M photos).
Keep its hard-won feature detectors. Replace only the final classification head.
Train that head on YOUR 500 images.

You're not training a model — you're adapting one.

The Honest Result

Transfer learning isn't free — domain match matters

ImageNet (colour photos) → Fashion-MNIST (grayscale silhouettes): big domain gap

Approach (2K imgs, head-only or fine-tune)	Test accuracy
TinyCNN from scratch	0.787
ResNet18 transfer — head only	0.715 ← loses!
ResNet18 — fine-tune layer4	0.832 ← wins

Lesson: transfer learning isn't magic. On a wide domain gap, head-only feature extraction can lose. Fine-tune the last conv block at a tiny LR to recover.
For Marcus's real catalogue (colour photos, closer to ImageNet), expect a much smaller gap.

Sarah's Friday Recommendation

Production-ready transfer learning

Sarah, end of L08:

"Start with a pretrained ResNet18. Train new head for a few epochs. Unfreeze the last conv block (layer4) at LR 1e-4. Fine-tune until val flattens. Expect to land in the low-90s top-1 on the 10 main categories with 500 photos per class — projected, since this colour catalogue sits much closer to ImageNet than Fashion-MNIST."

Recipe

1. Load pretrained ResNet18 backbone
2. Replace fc head → 10-class linear layer
3. Train head for 3 epochs at LR 1e-3
4. Unfreeze layer4, fine-tune at LR 1e-4 until validation curve flattens

Three Things To Remember

L08 takeaways

1

Convolutions exploit locality and translation invariance

Same kernel slides everywhere → fewer params, better inductive bias for images.

2

The PyTorch training loop doesn't change

Conv2d / MaxPool2d are drop-in replacements for Linear. L07's loop just works.

3

Don't train CNNs from scratch on small data

Use a pretrained backbone. Head-only sometimes works, fine-tuning usually does.

Method Cheat Sheet

Pick the right tool for the data you have

Scenario	Recommendation
Tabular features (rows × cols)	Stick with gradient boosting or LR (L02-L04)
10K+ small images, 10 classes	Train a small CNN from scratch
< 1K images per class	Transfer learning with a pretrained backbone
100+ classes, lots of data	Fine-tune a larger pretrained model
Production: need fast inference	MobileNetV3 backbone, or ONNX-compiled ResNet18
Need it Tuesday	Call a hosted vision API. Revisit later.

Bridge to L09

From pixels to words

Marcus, after the auto-tagger ships:

"Can we also do natural-language search?

A customer types 'blue summer dress' — surface the right products."

Next modality: text

L09 — Embeddings & NLP foundations

Turning words into vectors. Tokenisation, word embeddings, similarity search.

L10 — Transformers & GenAI

The architecture behind GPT, BERT, ChatGPT. Attention, generation, fine-tuning.

Q&A · Assignment Preview

What's due before L09

Assignment

CIFAR-10 classifier (10 categories of small colour photos):

Part A — CNN from scratch, beat 55% accuracy (stretch: 65%)

Part B — Transfer learning, beat 75% accuracy

Part C — Reflect on why transfer behaves differently here than on Fashion-MNIST

Optional self-study

- `optional_extensions.ipynb` — data augmentation, BatchNorm, learning-rate schedulers
- Try a different backbone (MobileNetV3 / EfficientNet) on the assignment
- Build a confusion matrix for the CIFAR-10 model — which pairs of classes get mixed up?

Next week: L09 — Embeddings & NLP foundations. See you Monday.